Linked Lists & HW1 Review

COMS 2132 — Lab 5

Today's Agenda

HW1 Review

- Written 2.1-2.6
- Wordle: top-down design

Linked Lists

- Linked Lists Basics (refresher)
- Examples

PART 1

HW1 Review

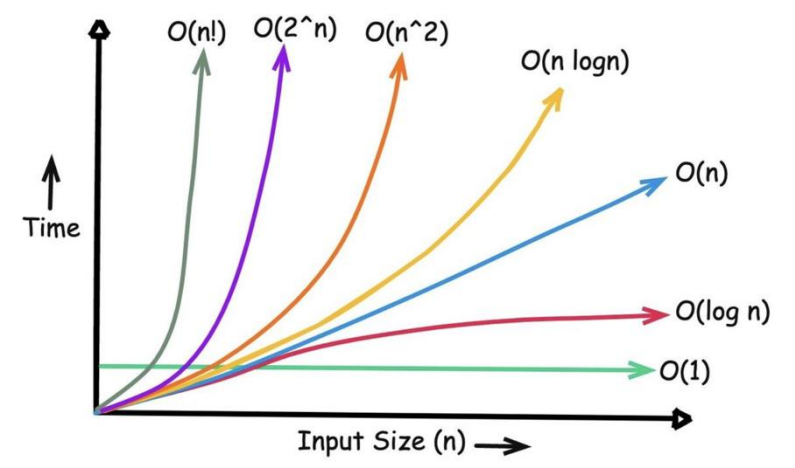
Written 2.1–2.6 & Wordle code walkthrough

2.1: Order of Growth

Note:
$2^{N+1} = 2 \cdot 2^N = O(2^N)$

In increasing order of their asymptotic growth rate:

128	→	$\log N$	→	$\sqrt{N}$	→	$3N$	→	$N \log N$	→	N^2	→	N^3	→	2^N / 2^{N+1}	→	4^N	→	$N!$
Constant O(1)		Logarithmic		$N^{1/2}$		Linear O(N)		N log N		quadratic		cubic		exponential		exponential		factorial



2.2: Double Exponential Tribute

The rulers of an ancient kingdom demanded tribute payments from their vassals each year. The first year, they required 2 sacks of grain. Each subsequent year, the tribute was squared. The second year, the vassals had to pay 4 sacks of grain, then 16, then 256, then 65536, ...

(a) What would the tribute be in year N?

Year 1: 2
Year 2: 2^2
Year 3: $(2^2)^2 = 2^4$
Year 4: $(2^4)^2 = 2^8$

$$\boldsymbol{D = 2^{(2^{(N-1)})}}$$

Double exponential growth.

(b) How many years would it take for the annual tribute payment to reach D sacks of grain?

Solve the equation in (a) for N.

$$\boldsymbol{D = 2^{(2^{(N-1)})}}$$

$$N - 1 = log_2 log_2 D$$

$$\boldsymbol{N = O(\log\log D)}$$

2.3: Compare 8N log N vs 2N²

The number of operations executed by algorithm A is $8N\log N$ and the number of operations executed by algorithm B is $2N^2$. Determine N_0 such that A is better than B for $N \geq N_0$. Show your solution path.

Start from: $8N \log_2 N \leq 2N^2$

Simplify: $4 \log_2 N \leq N$

Test:

$N = 8$
$\log_2 8 = 3$
$8/4 = 2 \rightarrow 3 \leq 2$ ❌

$N = 16$
$\log_2 16 = 4$
$16/4 = 4 \rightarrow 4 \leq 4$ ✓

Therefore: $N_0 = 16$

💡 Eventually N grows faster than log N, so N log N beats N^2 after some threshold.

2.4: print_duplicates — $O(N^3)$

```
def print_duplicates(li):

  duplicates = []

  for i in range(len(li)):                          # A
    for j in range(i+1, len(li)):                   # B
      if li[i] == li[j]:                            # C

      seen = False                                  # D
      for k in range(len(duplicates)):              # E
        if duplicates[k] == li[i]:                  # F
          seen = True
      if not seen:
        duplicates.append(li[i])
        print(li[i])
```

Line-by-Line Runtime

Line A: for i	N iterations
Line B: for j	up to N iterations
A × B together	$= O(N^2)$
Line E: for k	scans duplicates[]
Worst case	duplicates has O(N)
A × B × E	$O(N^2) \times O(N)$
Total	$= O(N^3)$

Answer: $O(N^3)$

2.5: print_duplicates2 — Sorted First

```
def print_duplicates2(li):

  li = sorted(li)                                  # A

  duplicates = []

  for i in range(1, len(li)):                      # B
    if li[i] == li[i-1]:                           # C

      seen = False                                 # D
      for k in range(len(duplicates)):             # E
        if duplicates[k] == li[i]:
          seen = True
      if not seen:
        duplicates.append(li[i])
        print(li[i])
```

Line-by-Line Runtime

Line A: sorted()	**O(N log N)**
Line B: for i	O(N) iterations
Line C: adjacent	O(1) per check
Line E: for k	scans duplicates[]
Worst case	O(N) entries
B × E	**O(N) × O(N) = O(N^2)**
Total	**= ~~O(N log N)~~ + O(N^2)**

Answer: O(N^2)

Improvement: Sorting eliminates the nested j loop (only adjacent checks now). But the duplicate-check loop for k is still O(N), making **O(N^2)** the **dominant term**.

2.6: Further Optimization

Skip Duplicates — O(N log N)

Sorted list → duplicates are adjacent.
Skip copies after printing once.

```
def print_duplicates3(li):

  li = sorted(li)          # O(N log N)

  i = 1
  while i < len(li):       # O(N) total
    if li[i] == li[i-1]:   # duplicate
      print(li[i])

      # skip all other copies
      while (i < len(li)-1 and
             li[i] == li[i+1]):
        i += 1

    i += 1
```

Both whiles combined = O(N) (i only moves forward)
Sorting dominates, so O(N log N)

Set Approach — Expected O(N)

Use a set for O(1) avg lookups.
No sorting needed, single pass.

```
def print_duplicates4(li):

  seen = set()
  duplicates = set()

  for item in li:          # O(N)
    if item in seen:       # O(1) avg
      if item not in duplicates:
        duplicates.add(item)
        print(item)
    seen.add(item)
```

Expected O(N) — set add/lookup are O(1) average

Wordle — Top-Down Design

README required: top-down ("wishful programming") design with 5+ incremental commits.

wordle() — Main Game Loop

```
def wordle():
  wordlist = read_wordlist(WORDLIST_FILE)
  word = random.choice(wordlist)
  guesses = 1

  while guesses <= 6:
    guess = user_guess()

    if guess in wordlist:
      print_word(word, guess)
      print(f"Guesses: {guesses}/6")
      if guess == word:
        print("Congratulations!")
        return
      else:
        guesses += 1
    else:
      print("Not in wordlist")

  print(f"The word was {word}")
```

Helper Functions

```
def read_wordlist(filename):
  words = []
  with open(filename, 'r') as f:
    for line in f:
      words.append(line.strip())
  return words

def user_guess():
  word = ''
  while len(word) != 5:
    word = input("5-letter word:")
  return word.lower()
```

Top-Down Steps (from README)

1. Write rules of the Wordle game
2. Design main function structure
3. Break into sub-functions
4. Implement function bodies
5. Test the whole program

Wordle: print_word — The Corner Case (Extra)

If the word has a letter once but the guess has it twice (one green, one wrong position) — the wrong one should be gray, not yellow.

```
def print_word(word, guess):
  letter_count = defaultdict(int)
  correct = set()
  for letter in word:
    letter_count[letter] += 1    #{'w':1,'o':1,…}

  # Pass 1: find greens (exact matches)
  for i in range(len(guess)):
    if word[i] == guess[i]:
      correct.add(i)
      letter_count[word[i]] -= 1

  # Pass 2: assign colors
  for i in range(len(guess)):
    if i in correct:
      print_green(guess[i])
    elif letter_count[guess[i]] > 0:
      print_yellow(guess[i])
      letter_count[guess[i]] -= 1
    else:
      print_gray(guess[i])
```

Two-Pass Strategy

Pass 1: Greens

Find exact matches first.
Decrement letter_count for each green so it's not double-counted as yellow.

Pass 2: Yellow / Gray

For non-green positions:
- count > 0 → yellow (letter exists)
- else → gray

Decrement on yellow to prevent over-marking duplicate letters.

PART 2

Linked Lists

Structure, operations & reference manipulation

Why Linked Lists?

Python's built-in list (array-based) has some shortcomings:

- Inserting/removing in the middle or beginning is expensive — O(n)
- Array size could be 2× the number of elements (wasted memory)
- Amortized O(1) for append may be unacceptable for real-time apps

Linked List Solution

- Does NOT use contiguous memory — nodes can be anywhere in memory
- Each node stores a reference to the element and a pointer to the next node
- Efficient O(1) insertions/deletions at known positions

Singly Linked List — Structure

Key Terminology

Node — Object containing an element + reference to next node

Element — The actual data stored (any Python object)

Head — First node in the list

Tail — Last node (its next is None)

```
class Node:
    def __init__(self, element, next=None):
        self.element = element
        self.next = next

class SinglyLinkedList:
    def __init__(self):
        self.head = None
```

Singly Linked List — Operations

add_first(element) — O(1)

1. Create a new node
2. Set new node's next → current head
3. Update head → new node
4. If list was empty, also set tail

```
def add_first(self, element):
    new_node = Node(element,
                    self.head)
    self.head = new_node
    if self.tail is None:
        self.tail = new_node
    self.size += 1
```

remove_first() — O(1)

1. If head is None, raise error
2. Save head's element
3. Set head → head.next
4. If list is now empty, set tail = None

```
def remove_first(self):
    if self.head is None:
        raise Empty("List empty")
    el = self.head.element
    self.head = self.head.next
    if self.head is None:
        self.tail = None
    self.size -= 1
    return el
```

Stacks & Queues from Linked Lists

LinkedStack (LIFO)

```
class LinkedStack:
    def __init__(self):
        self._data = SinglyLinkedList()
    def push(self, el):
        self._data.add_first(el)
    def pop(self):
        return self._data.remove_first()
```

LinkedQueue (FIFO)

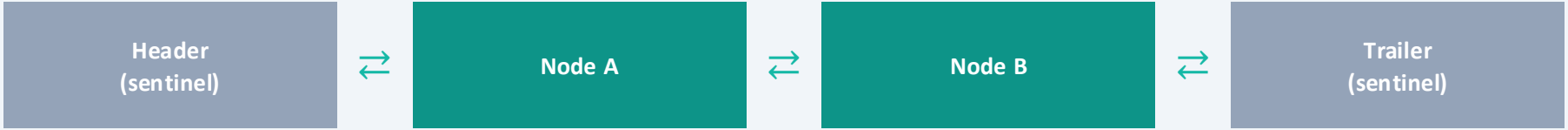
```
class LinkedQueue:
    def __init__(self):
        self._data = SinglyLinkedList()
    def enqueue(self, el):
        self._data.add_last(el)
    def dequeue(self):
        return self._data.remove_first()
```

Key Insight

Stack uses add_first / remove_first (both at head).
Queue uses add_last / remove_first (enqueue at tail, dequeue at head).
All operations are O(1) worst-case.

Doubly Linked Lists

Singly linked lists can only traverse forward. A doubly linked list adds a prev pointer to each node, enabling bidirectional traversal and efficient deletions from either end.



Sentinels (Header & Trailer)

- Fixed empty nodes — the list is never truly empty
- Every node has both prev and next (no null checks)
- Insertions always happen between two existing nodes
- Greatly simplifies add/remove logic

```
class Node:
    def __init__(self, element,
                 prev, next):
        self.element = element
        self.next = next
        self.prev = prev  # NEW!


# Init with sentinels
   header.next = trailer
   trailer.prev = header
```

DLL — Insert & Delete Operations

_insert_between() — O(1)

Insert a new node between any two adjacent nodes. With sentinels, this covers all cases uniformly.

```
def _insert_between(self, e, predecessor,
                    successor):
    new_node = Node(e, predecessor, successor)
    predecessor.next = new_node
    successor.prev = new_node
    self.size += 1
    return new_node
```

_delete_node() — O(1)

Remove a node by linking its neighbors to each other. Sentinels guarantee both neighbors exist.

```
def _delete_node(self, node):
    el = node.element

    predecessor = node.prev
    successor = node.next

    successor.prev = node.prev
    predecessor.next = node.next
    self.size -= 1

    return el
```

Arrays vs. Linked Lists — Performance

Operation	Linked List	Array (Python list)
push(e) / add_first	**O(1)** worst-case	O(n) worst-case
pop() / remove_first	**O(1)** worst-case	O(n) worst-case
append / add_last	**O(1)** worst-case	amortized **O(1)**
Access by index [i]	O(n)	**O(1)**
Insert at position k	**O(1) if ref known, else O(n)**	**O(n)**
Memory overhead	Higher (prev/next pointers)	**Lower** (contiguous)

Takeaway: Use **arrays** for random access. Use **linked lists** when you need guaranteed O(1) insertions/deletions at known positions and can traverse sequentially.

PART 3

Practice

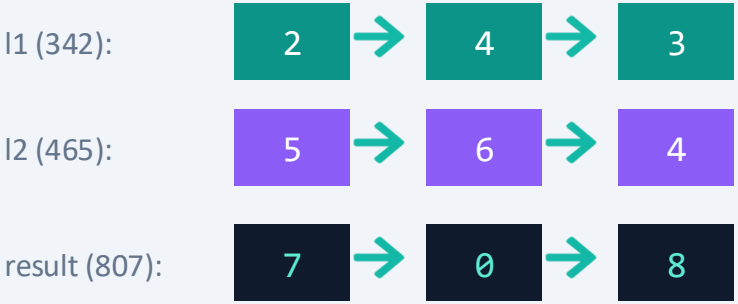
Applying linked list concepts to examples

Example #1: Add Two Numbers

Medium

Two non-empty linked lists represent non-negative integers. Digits are stored in reverse order. Each node contains a single digit. Add the two numbers and return the sum as a linked list.

Example: 342 + 465 = 807



Key Insight

Digits are already in reverse order — the ones place is first! This means we can add digit-by-digit from head to tail, just like manual addition from right to left, carrying over as we go.

Approach 1: Brute Force

Convert to integers, add, convert back

Approach 2: Optimal

Digit-by-digit addition with carry

Example #1: Brute Force Approach

Naive

```
def addTwoNumbers(l1, l2):
    num1, num2, mult = 0, 0, 1
    while l1:
        num1 += l1.val * mult
        mult *= 10;  l1 = l1.next          # Step 1: Convert
    mult = 1                               to integers
    while l2:
        num2 += l2.val * mult
        mult *= 10;  l2 = l2.next

    total = num1 + num2                    # Step 2: Add

    dummy = ListNode(0)
    cur = dummy
    if total == 0: return dummy
    while total > 0:                       # Step 3: Convert
        cur.next = ListNode(total % 10)    back to linked list
        total //= 10
        cur = cur.next
    return dummy.next
```

3-Step Process

1. Traverse lists → build integers
2. Add integers normally
3. Decompose sum → new list

Drawbacks

- Integer overflow risk in typed languages
- 3 full traversals instead of 1
- Doesn't leverage reversed digits
- **Interviewers want the carry approach**

Time: O(n + m) | **Space: O(max(n, m))** | Works, but not optimal (for interviews)

Example #1: Optimal Solution

Optimal

Add digit-by-digit with carry — no integer conversion needed

```
def addTwoNumbers(l1, l2):
    dummy = ListNode(0)
    cur = dummy
    carry = 0

    while l1 or l2 or carry:
        v1 = l1.val if l1 else 0
        v2 = l2.val if l2 else 0

        total = v1 + v2 + carry
        carry = total // 10       # carry for next digit
        digit = total % 10        # current digit

        cur.next = ListNode(digit)
        cur = cur.next
        l1 = l1.next if l1 else None
        l2 = l2.next if l2 else None

    return dummy.next
```

No integer conversion needed. Handles arbitrarily large numbers. Processes each digit exactly once.

Time: **O(max(n, m))** — single pass
Space: **O(max(n, m))** — result list

Example #1: Step-by-Step Walkthrough

`Input: l1 = [2,4,3]  l2 = [5,6,4]  (representing 342 + 465)`

Step	l1.val	l2.val	carry in	total	digit	carry out
1	2	5	0	7	7	0
2	4	6	0	10	0	1
3	3	4	1	8	8	0

Result linked list: 7 → 0 → 8 = 807

Brute Force vs. Optimal

	Brute Force	Optimal (Carry)
Approach	Convert → add → convert	Add digit-by-digit
Integer overflow?	Yes (in some languages)	Never
# of passes	3 passes	1 pass
Interviewer preferred?	No — misses the point	Yes — shows understanding

Example #2: remove_all - remove(value)

Implement a simple “remove(value)” method, similar to remove_first() but specifies a certain value (element) to be removed from a singly linked list.

Basic singly linked list setup:

```
class Node:
    def __init__(self, value):
        self.value = value
        self.next = None


class LinkedList:
    def __init__(self):
        self.head = None

    def append(self, value):
        new_node = Node(value)
        if not self.head:
            self.head = new_node
            return

        current = self.head
        while current.next:
            current = current.next
        current.next = new_node
```

```
def remove(self, value):
    current = self.head
    prev = None

    while current:
        if current.value == value:
            if prev is None:
                # Removing head
                self.head = current.next
            else:
                prev.next = current.next
            return True  # removed successfully

        prev = current
        current = current.next

    return False  # value not found
```

To remove a node in a singly linked list:

- We must find and keep track of the previous node.
- Then we “skip” the node by updating pointers.

Example #2: remove_all - Naive vs Efficient

Naive — $O(N^2)$

```
def remove_all_naive(self, value):
    while self.remove(value):
        pass
```

Why $O(N^2)$?

- Each remove() traverses from the head, removes one element, and starts over
  - Each remove is up to O(N)
  - If there are N duplicates, N + (N-1) + (N-2) + ...
- If O(N) duplicates → called O(N) times
- **$O(N) \times O(N) = O(N^2)$**
- **Restarts from head every time!**

Efficient — O(N)

```
def remove_all(self, value):
    while (self.head and
      self.head.value == value):
        self.head = self.head.next

    current = self.head
    prev = None

    while current:
        if current.value == value:
            prev.next = current.next
        else:
            prev = current
        current = current.next
```

Why O(N)?

- Single pass — no restart from head
- Each node visited exactly once
- **Total: O(N)**

Example #2: remove_all() - comparison

remove_all_naive() vs. remove_all()

	What it removes	When it stops	Runtime
remove_all_naive()	First occurrence only on repeat	After one removal, then restarts from the head	$O(N^2)$
remove_all()	Every occurrence	Traverses entire list	$O(N)$

Visual example - removing “3”:

remove_all_naive()

1 → 3 → 2 → 3 → 4 → 3 — Step 1 — O(N)

1 → 2 → 3 → 4 → 3 — Step 2 — O(N)

1 → 2 → 4 → 3 — Step 3 — O(N)

1 → 2 → 4 — Final: $O(N^2)$

remove_all()

1 → 3 → 2 → 3 → 4 → 3 — One-step — O(N)

1 → 2 → 4 — Final: O(N)